

Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering, Pune

Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)

PROJECT REPORT

Interview Prep Crew:
A Multi-Agent AI System for Interview Preparation and Resume ATS Scoring

Name	PRN
Tejas Shidam	123B1C037
Sandesh Phad	123B1C033

Guided by:
Prof. Santwana Gudadhe

Academic Year: 2026–2027

Abstract

Interview Prep Crew is an agentic web application that prepares a candidate for a job interview and checks a resume against applicant tracking system (ATS) rules. The candidate enters a company, a role and an experience level, and can add a job description and a resume. Four CrewAI agents then work one after another: the Company Researcher searches the web and news, the Role Analyst works out the skills and interview rounds, the Question Generator writes ten likely questions, and the Answer Coach drafts model answers and a five-day plan. A fifth agent, the ATS Resume Reviewer, turns a rule-based resume score into prioritised fixes and bullet rewrites.

The central design rule is that code scores and the language model writes. The ATS score out of 100 is computed by deterministic rules in six areas, so it is instant and repeatable. Every agent step, model call and tool call is streamed live to the browser through Server-Sent Events, which makes the work of each agent visible. A resume is used to personalise questions but is never saved to disk and is hidden from the activity log. The application is built with FastAPI, CrewAI 1.15, plain HTML, CSS and JavaScript, and pdfplumber, and runs with Groq (gpt-oss-120b) or a local Ollama model; it can also use Anthropic Claude.

On Groq the four-agent crew finished in 2 minutes 14 seconds and produced all ten questions and ten coached answers; a 4B local model on a laptop CPU needed about six minutes. The ATS engine scored a strong sample resume 98/100 and a weak one 4/100. All 155 automated tests pass, and one of them exposed a leak of resume text into saved history, which we fixed. The ATS score is a heuristic that has not been checked against commercial ATS software, and results depend on web-search quality and on the chosen model.

Keywords: Agentic AI, Multi-Agent Systems, CrewAI, Interview Preparation, Resume Screening, Applicant Tracking System, Tool-Augmented LLM, Server-Sent Events.

Table of Contents

Abstract 2

Chapter 1: Introduction & Problem Statement 3

Chapter 2: Literature Survey 3

Chapter 3: System Architecture and Methodology 4

Chapter 4: Tools and Technologies 5

Chapter 5: System Implementation 5

Chapter 6: Results and Performance Analysis 5

Chapter 7: Future Scope 6

Chapter 8: Conclusion 6

Chapter 9: References 6

Chapter 1: Introduction & Problem Statement

1.1 Introduction

Preparing for an interview means doing four separate jobs: researching the company, understanding what the role demands, guessing the questions and practising the answers. Candidates also need a resume that passes automated screening before a person reads it. We built Interview Prep Crew to do the research and drafting and to leave the judgement with the candidate. Each agent has one job and one set of tools, and its output is handed to the next agent.

1.2 Problem Statement

- **Scattered preparation.** Company facts, role requirements and interview experiences are spread over many sites.
- **Generic practice.** A single chatbot prompt gives generic questions that ignore the company's real facts and the candidate's own projects.
- **Opaque resume screening.** Candidates rarely learn why a resume fails an ATS: keyword gaps, missing sections and weak bullets stay invisible.
- **Unverifiable AI output.** Answers from one prompt carry no sources, so their facts cannot be checked.

This project asks: can a small team of narrowly scoped agents, using real search tools and a deterministic scoring engine, give a candidate sourced, personalised preparation that can be watched step by step?

1.3 Objectives

- Build a web application with a live view of every agent, tool call and hand-off.
- Research a company and role from web and news results, with source links.
- Generate ten likely questions and coached answers, personalised from the candidate's resume.
- Score a resume out of 100 across six ATS areas without calling a model, and add an AI review.
- Keep resumes private: never store them and keep them out of the activity log.
- Support Groq, Claude and a local model, and test everything automatically.

1.4 Scope of the Project

The system covers interview preparation and resume screening for one candidate at a time. It is a preparation aid, not a hiring tool: it does not apply for jobs, contact employers or rank candidates. Search relies on DuckDuckGo results, the ATS score is a heuristic, and the application has no user accounts and runs one AI crew at a time.

Chapter 2: Literature Survey

2.1 Tool-Using Language Models and Agents

Chain-of-thought prompting [1] showed that models reason better when they write intermediate steps. ReAct [2] interleaves that reasoning with actions such as search, which is the loop our research agents follow with their tools. Toolformer [3] shows that models can learn when to call tools, and retrieval-augmented generation [4] grounds answers in retrieved passages; our researcher agents apply the same idea with live web search instead of a fixed index.

2.2 Multi-Agent Frameworks

AutoGen [5] and MetaGPT [6] organise several model-driven agents around conversation or software-team roles. CrewAI [7] offers role-based agents, tasks and a sequential process in which later tasks receive earlier outputs as context, which suits our fixed pipeline.

2.3 Resume Screening

Commercial ATS vendors do not publish their scoring rules, and we found no public benchmark of ATS behaviour. The rules here therefore follow widely documented screening practice: standard headings, job-description keywords, contact details, quantified achievements, action verbs and sensible length. The result is a heuristic.

Table 2.1: Summary of reviewed works

Ref	Work or resource	Area	Used in this project for
[1]	Wei et al., chain-of-thought prompting	Prompting	Step-by-step task instructions
[2]	Yao et al., ReAct	Reasoning and acting	Tool-calling loop of the research agents
[3], [4]	Toolformer; retrieval-augmented generation	Tool use, grounding	Sourced answers from search results
[5], [6]	AutoGen; MetaGPT	Multi-agent systems	Role-based agent design
[7]	CrewAI	Agent framework	Agents, tasks, sequential process, event bus

2.4 Research Gap

These works each address one component: reasoning, tool use or an agent framework. A candidate needs them together in one workflow, with sources, personalisation, a deterministic resume score and a visible audit trail. The contribution of this project is that integration for interview preparation; it is not a claim that any single technique used here is new.

Chapter 3: System Architecture and Methodology

3.1 Architecture

The browser talks to a FastAPI server over REST and Server-Sent Events. The server starts a CrewAI crew in a background thread, and an event bridge subscribes to CrewAI's event bus and turns each agent, model and tool event into a small JSON event for the browser. The crew calls a language model (Groq, Claude or Ollama) and three tools. Code decides and the model writes: scores come from deterministic code, every step is shown live, and finished runs are saved as JSON files without resume text.

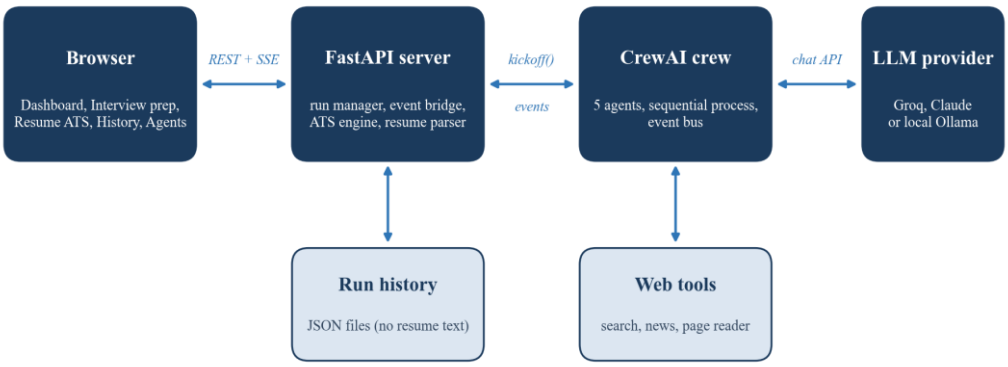


Figure 3.1: System architecture

3.2 Agents and Crew

Table 3.1: The five agents

Agent	Tools	Reads	Produces
Company Researcher	web_search, news_search	Company, role, date	Company brief with sources
Role Analyst	web_search, read_webpage	Brief, job description, resume	Skills, interview rounds, gaps
Question Generator	none (reasoning)	Brief, analysis, resume	10 likely questions
Answer Coach	none (reasoning)	Questions, brief, resume	Answers, tips, 5-day plan
ATS Resume Reviewer	none (reasoning)	Resume, job description, ATS score	Prioritised fixes and rewrites

The prep crew uses CrewAI's sequential process: each task lists all earlier tasks as context, so later agents work from real research instead of guessing. When a resume is supplied, the Role Analyst bases the gaps on it, the Question Generator tags at least three questions as "From your resume", and the Answer Coach builds answers on the candidate's own projects, using only facts the resume states.

3.3 ATS Scoring Method

Table 3.2: ATS scoring areas (100 points)

Area	Points	What is checked
Keyword match	35	Job-description keywords found in the resume (80% match earns full marks)
Sections	15	Education, skills, experience or projects, summary, certifications
Contact info	10	Email, phone, LinkedIn or GitHub link
Impact	15	Bullets that include numbers (target 30%) and start with an action verb (target 60%)
Formatting	15	Length of 350–800 words, dates, consistent bullets, no walls of text
Language	10	Weak phrases such as "responsible for", first-person pronouns, repeated openers

Keywords come from the job description: known skills first, then repeated meaningful words. Without a job description the keyword area is skipped and the rest is rescaled to 100. The reviewer agent receives the score as ground truth, may only rewrite bullets that exist in the resume, and writes [add metric] instead of inventing a number.

Chapter 4: Tools and Technologies

Table 4.1: Technology stack

Layer	Technology	Role in the system
Agent framework	CrewAI 1.15 [7]	Agents, tasks, sequential crew, event bus
Language models	Groq gpt-oss-120b [10], [11]; Anthropic Claude; Ollama qwen3:4b-instruct [12]	Writing and tool calls; local fallback
Search tools	ddgs (DuckDuckGo) [14], httpx	Web search, news search, page reader
Backend	Python 3.13, FastAPI [8], Uvicorn	REST API, event stream [9], run manager
Resume parsing	pdfplumber [13], zip and XML	Text from PDF, DOCX, TXT and MD files
Frontend	HTML, CSS, JavaScript (no libraries)	Dashboard, live agent view, ATS pages
Storage	JSON files	Run history (no resume text)
Testing	pytest	155 automated tests with a scripted fake model; run on a Windows 11 laptop

Chapter 5: System Implementation

5.1 Backend and Live Events

A run manager starts each crew in a background thread and keeps one ordered event list per run. The event bridge translates CrewAI events (agent started, model call, streamed text, tool call, task completed) and the stream endpoint sends them to the browser, resuming from the last event id after a reload. Only one crew runs at a time because CrewAI's event bus is process-wide.

5.2 Frontend

The interface has five pages: a Dashboard (statistics, recent runs, score trend), Interview prep (live agent cards with Activity, Prep pack and Sources tabs), Resume ATS (upload or paste, score gauge, breakdown, keywords, AI review), History (search, download, delete) and Agents (roles, tools, architecture). It has light and dark themes and works on phones.

5.3 Reliability and Privacy

- Search retries with shorter queries when nothing is found, and a failed tool returns a message instead of crashing the run.
- A reply that ends without a finish reason or token usage (for example a local model out of memory) is flagged as cut off.
- Groq rate limits are retried, the resume is redacted from task-prompt events, and a launcher picks the next free port if 8010 is busy.

Chapter 6: Results and Performance Analysis

We ran the crew for "Software Engineer at Infosys, fresher" on each model and scored sample resumes against a Backend Engineer job description. Timings were measured by the application itself.

Table 6.1: Crew run time by model

Setup	Total time	Notes
Groq, gpt-oss-120b	2 min 14 s	Researcher 23 s, Analyst 39 s, Question Generator 39 s, Answer Coach 31 s; 10 questions and 10 answers
Ollama, qwen3:4b-instruct (CPU)	about 6 min (349 s, 392 s)	Complete output; the Answer Coach step alone took about 2 min 20 s
Ollama, qwen3-vl:4b (thinking model, CPU)	over 21 min for 3 of 4 agents	Researcher 6 min 31 s, Analyst 4 min 24 s, Question Generator 10 min 38 s; run abandoned

Table 6.2: ATS scores of sample resumes

Sample resume	Words	ATS score	Grade	Job keywords found
Strong resume, with job description	305	98	Excellent	11 of 13
Strong resume, no job description	305	97	Excellent	not scored
Short resume with weak bullets	82	35	Weak	1 of 13
Paragraph with no headings or contact details	80	4	Weak	0 of 13

6.1 Observations

- The ATS score ordered the samples correctly in milliseconds. The AI review on Groq took about 2 seconds and rewrote only existing bullets, writing [add metric] where a number was unknown.
- All 155 automated tests pass. They run the real crew with a scripted fake model and search, covering tools, events, SSE, ATS scoring, PDF and DOCX parsing, history and privacy.
- A privacy test found resume text in saved task prompts; it is now redacted before events are emitted.
- Groq's free tier allows about 8,000 tokens per minute, and a 27B model on it rejected longer answers (1,000 output tokens per minute), so gpt-oss-120b became the default.

6.2 Limitations

The ATS score is a heuristic and was not compared with commercial ATS software. DuckDuckGo search can be rate-limited, and small local models write weaker answers (one local run was cut off when Ollama ran out of memory, which is why cut-off replies are now flagged). The Claude option was tested only with mocked requests, and the system has not been tested with real candidates.

Chapter 7: Future Scope

- An interactive mock interview that asks a question, scores the answer and follows up.
- An evaluation script that compares models across many companies on quality, time and cost.
- A fact-checker agent that flags brief claims not supported by the sources.
- A user study with students, and a check of the ATS rules against labelled resumes.
- User accounts, Docker deployment and automatic fallback between models.

Chapter 8: Conclusion

Interview Prep Crew shows that a small team of narrowly scoped agents can research a company, analyse a role, predict questions and coach answers while every step stays visible to the user. Deterministic code scores the resume, the model only writes, and the resume is never stored. On Groq a full preparation takes about two minutes, and 155 automated tests keep the system dependable. The ATS rules are unvalidated, results depend on search and model quality, and no real users have tried it yet; an evaluation script and a user study are the next steps toward a dependable preparation assistant.

Chapter 9: References

[1] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[2] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[3] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[5] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[6] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in Proc. ICLR, 2024.

[7] CrewAI, Inc., "CrewAI: Framework for orchestrating role-playing, autonomous AI agents," GitHub repository and documentation, 2025.

[8] S. Ramírez, "FastAPI," documentation, https://fastapi.tiangolo.com.

[9] WHATWG, "HTML Living Standard: Server-sent events."

[10] OpenAI, "gpt-oss-120b and gpt-oss-20b model card," 2025.

[11] Groq, Inc., "GroqCloud API documentation," https://console.groq.com/docs.

[12] Ollama, "Ollama: Run large language models locally," https://ollama.com.

[13] J. Singer-Vine, "pdfplumber," GitHub repository.

[14] "ddgs: A metasearch library for DuckDuckGo and other engines," Python Package Index, 2025.